Лабораторна робота № 4–ЕС
Зовнішні функції в середовищі CLIPS

Мета роботи – вивчити особливості реалізації зовнішних функцій у середовищі CLIPS для створення ЕС.

Теоретичні відомості

У CLIPS існують *внутрішні* і *зовнішні* функції. Внутрішні функції реалізовані середовищем CLIPS, тому їх можна використовувати в будь-який момент. *Зовнішні функції* – це функції, написані користувачем. Процедурний механізм дозволяє користувачеві за допомогою вбудованих в мову функцій розробляти або конструювати нові функції, які виконують деякі корисні дії або повертають деякі значення. У цьому сенсі CLIPS нагадує такі відомі мови програмування, як C, C++ або Pascal. Так, для створення зовнішніх (користувацьких) функцій використовується конструктор ***deffunction,*** який має такий синтаксис:

(deffunction ім'я_функції
[необов'язковий коментар]
(список формальних параметрів)
(дія_1)
(дія_2)
.....
(дія_N)

Виклик функції здійснюється по імені, заданим користувачем. За ім'ям функції слідує список необхідних аргументів, відділений одним або більшою кількістю пробілів. Виклик функції разом зі списком аргументів повинен бути взятий у дужки. Послідовність дій, що визначена за допомогою конструктора функції ***deffunction,*** виконується інтерпретатором CLIPS.

Синтаксис конструктора deffunction містить п'ять елементів:

- ім'я функції ;
- необов'язкові коментарі;
- список з нуля або більше параметрів;
- необов'язковий символ групових параметрів для зазначення того, що функція може мати змінне число аргументів;
- послідовність дій або виразів, які будуть виконані (обчислені) по порядку в момент виклику функції.

Функція, створювана за допомогою конструктора ***deffunction,*** повинна мати унікальне ім'я, що не збігається з іменами інших зовнішніх і внутрішніх функцій. Конструктор ***deffunction*** має бути оголошений до першого використання створюваної ним функції.

Наприклад, визначимо функцію *om(x, y),* яка повертає цілу частину частки від ділення змінної *y* на змінну *x*:

```
(deffunction om
        (?x ?y)
        (div ?y ?x))
```

Зверніть увагу на те, що у CLIPS ім'я змінної починається з символу **"?"**, що для виклику функції (у даному випадку внутрішньої функції ділення націло div) використовується префіксна нотація.

<u>Приклад.</u> Нехай потрібно підібрати резистор для ділянки кола схеми електричної принципової деякого радіоелектронного пристрою. Резистор характеризується опором, який визначається за виміряним або розрахованим значенням електричного струму, що проходить через резистор, і падінням напруги на ньому. Програма з ім'ям *resistor.clp*, що вирішує цю задачу, може бути, наприклад, такою:

```
(deffacts resistors          ; This is database
    (resistor Ra 2)
    (resistor Rb 5)
    (resistor Rc 7))
(deffunction om              ; This is function om(x,y)
    (?x ?y)
    (div ?y ?x))

(defrule input               ; This is current & strait input
    (initial-fact)
    =>
    (printout t crlf “Input current value: “)
    (bind ?i (read))
    (printout t “Input strait value: “)
    (bind ?u (read))
    (assert (numbers ?i ?u)))

(defrule take                ; Get resistor from database
        (numbers ?i ?u)
        (resistor ?r =(om ?i ?u))
        =>
        (printout t crlf “You must take resistor “ ?r”.” crlf crlf)
        (reset)
        (halt))

(defrule nothing             ; There is nothing
    (numbers ?i ?u)
    (resistor ?r ~=(om ?i ?u))
    =>
    (printout t crlf “There is nothing for You in my database!”
```

```
 crlf crlf)
(reset)
(halt))
```

Програма складається з декількох частин: БД з ім'ям *resistors*, оголошення зовнішньої функції *om* і трьох правил з іменами *input, take, nothing*.

У БД містяться відомості про резистори. Вони подані у вигляді списків, які складаються з трьох полів. Перше поле має значення *resistor*, яке відображує тип радіодеталі. У другому полі списку міститься тип резистора. Останнє поле зберігає значення опору.

Правило ***input*** призначене для введення вихідних даних. Воно активізується системним початковим фактом і потребує від користувача ввести струм і напругу. Вбудована функція ***read*** повертає значення, введене зі стандартного пристрою введення (клавіатури), яке зберігається у змінних *?i* і *?u* . У правій частині правила виконується ще одна дія. Команда ***assert*** додає у робочу пам'ять інтерпретатора CLIPS факт *(numbers ?i ?u)* для того, щоб можна було звертатися до локальних змінних *?i* і *?u*, пов'язаних з правилом *input* , з інших правил програми.

У наступних двох правилах користувачеві або пропонується тип підходящого резистора (правило *take*), або повідомляється, що такий відсутній (правило *nothing*).

Ліва частина правила ***take*** складається з двох умов, тому правило активізується, якщо обидві умови будуть задоволені. Перша умова задовольняється, оскільки відповідний факт уже створений правилом *input*. Друга умова задовольниться, якщо буде точно відповідати будь-якому факту (списку) у БД. У другому полі умови знаходиться змінна *?r*, яка може прийняти значення *Ra*, або *Rb*, або *Rc* залежно від вмісту третього поля умови. У цьому полі здійснюється виклик функції *om* і зберігається повертане функцією значення. Так, якщо повернене значення дорівнює 7, то умова задовольниться, змінна *?r* прийме значення *Rc*, правило активізується і виведе на екран монітора пропозицію вибрати резистор *Rc*. Якщо повертане функцією *om* значення дорівнює 5, то користувачеві буде запропоновано резистор *Rb* і т.д.

У лівій частині правила ***nothing*** начебто повна аналогія, за винятком однієї маленької модифікації. У третьому полі другої умови перед викликом функції *om* стоїть символ **"~"**, що означає логічне заперечення. Таким чином, умова задовольниться і правило активізується, якщо повертане функцією *om* значення буде не 2, не 5 і не 7.

Перебуваючи в інтерпретаторі CLIPS, командою ***(clear)*** очистимо його від даних попереднього прикладу, завантажимо файл *resistor.clp* і запустимо програму на виконання. У результаті чого отримаємо щось подібне:

```
CLIPS> (clear)
CLIPS> (load resistor.clp)
. . . . . . . . . . . . .
TRUE
CLIPS> (reset)
CLIPS> (run)

Input current value: 3
Input strait value: 15

You must take resistor Rb.

CLIPS> (run)

Input current value: 3.5
Input strait value: 5.44

There is nothing for You in my database!

CLIPS>
```

Таким чином, файл *resistor.clp* також являє собою БЗ,оскільки містить і БД, і відомості (правила) про те, як дані можуть бути використані. Ця база має у своєму розпорядженні принаймні три знання. Перше знання – загальний список резисторів із зазначенням типу і опору, друге знання – закон Ома, третє знання – пропонований тип резистора.

Інтелект БЗ можна істотно підвищити додаванням нових даних і правил. Так, замість закону Ома можна використовувати більш серйозні методики визначення опору резистора. Інший шлях – додавання нових типів резисторів (*Rd*) в БД. Можна ще додати правило (правила), яке вибере з резисторів *Ra* і *Rd* переважний для деяких конкретних умов і т.д. і т.п.

Порядок виконання роботи

1. Реалізувати наведений приклад у середовищі CLIPS.
2. Протестувати отриману ЕС.

Звіт про виконання роботи

Звіт про виконання лабораторної роботи № 4–ЕС має містити:

1. Титульний аркуш.
2. Програмний код, що складається з необхідного набору команд і функцій, а також результатів їх виконання для кожного пункту даної лабораторної роботи.
3. Екранні форми, що показують результати виконання кожного пункту даної лабораторної роботи.
4. Висновки.